

Modul 5 NLP - Week #6 - Transformer dan BERT

February 26, 2026

1 Modul 5 - NLP - Transformer dan BERT

Perkembangan NLP mengalami lompatan besar sejak diperkenalkannya arsitektur Transformer oleh Vaswani et al. (2017). Transformer mengatasi keterbatasan RNN/LSTM dengan mekanisme self-attention yang memungkinkan pemrosesan paralel dan pemahaman konteks global.

BERT (Bidirectional Encoder Representations from Transformers) merupakan implementasi Transformer berbasis encoder yang menghasilkan contextual embedding, sehingga mampu memahami makna kata berdasarkan konteks kalimat secara dua arah (bidirectional).

Pada modul ini mahasiswa akan:

- 1) Memahami konsep self-attention dan arsitektur Transformer
- 2) Mengimplementasikan pre-trained BERT
- 3) Melakukan fine-tuning sederhana untuk klasifikasi teks
- 4) Menganalisis contextual embedding
- 5) Membandingkan embedding statis vs contextual

1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- 1) Memahami konsep dasar Transformer dan mekanisme self-attention.
- 2) Menjelaskan perbedaan embedding statis dan contextual embedding.
- 3) Mengimplementasikan pre-trained BERT menggunakan HuggingFace.
- 4) Menghasilkan embedding kata dan kalimat menggunakan BERT.
- 5) Melakukan fine-tuning BERT untuk klasifikasi teks.
- 6) Mengevaluasi performa model menggunakan metrik klasifikasi.

1.2 Alat dan Bahan

Perangkat Keras

- Laptop/PC dengan spesifikasi minimal RAM 4 GB.

Perangkat Lunak

- Python (versi 3.7 ke atas)
- Jupyter Notebook atau Google Colab
- Library Python yang diperlukan:

- 1) transformers
- 2) torch

- 3) numpy
- 4) pandas
- 5) scikit-learn
- 6) matplotlib
- 7) seaborn

1.3 Materi

Sebelum Transformer diperkenalkan, model yang dominan dalam Natural Language Processing (NLP) adalah Recurrent Neural Network (RNN) dan variannya seperti LSTM (Long Short-Term Memory) dan GRU (Gated Recurrent Unit). Model-model ini dirancang untuk memproses data sekuensial, seperti teks, dengan membaca kata satu per satu secara berurutan.

Namun, terdapat dua kelemahan utama pada pendekatan tersebut:

- 1) Sulit menangkap dependensi jarak jauh (long-range dependency): Informasi pada awal kalimat sering kali “hilang” ketika model memproses bagian akhir kalimat.
- 2) Tidak dapat diparalelkan secara efisien: Karena sifatnya sekuensial (kata ke-kata), proses pelatihan menjadi lambat dan tidak optimal untuk komputasi GPU modern.

Untuk mengatasi keterbatasan ini, pada tahun 2017, Vaswani et al. memperkenalkan arsitektur baru melalui paper:

- Attention Is All You Need

Arsitektur ini disebut Transformer.

1.3.1 1. Apa itu Transformer?

Transformer adalah arsitektur deep learning berbasis mekanisme self-attention yang dirancang untuk memproses data sekuensial tanpa menggunakan struktur rekuren (tanpa RNN).

Karakteristik utama Transformer:

- 1) Tidak menggunakan recurrence
- 2) Menggunakan self-attention sebagai mekanisme utama
- 3) Dapat diproses secara paralel
- 4) Lebih cepat dilatih dibanding RNN/LSTM
- 5) Mampu menangkap hubungan kata jarak jauh secara efektif

Transformer menjadi fondasi berbagai model modern seperti: 1) BERT 2) GPT 3) RoBERTa 4) T5 5) LLaMA

1.3.2 2. Konsep Utama: Attention

Attention secara sederhana berarti:

- Kemampuan model untuk menentukan bagian mana dari input yang paling penting.

Analogi sederhana: Ketika membaca kalimat panjang, manusia secara alami fokus pada kata-kata penting untuk memahami makna. Transformer meniru mekanisme ini melalui attention.

1.3.3 3. Self-Attention Mechanism

Self-attention memungkinkan setiap kata dalam kalimat:

- 1) “Melihat” semua kata lain
- 2) Menghitung seberapa penting kata lain terhadap dirinya
- 3) Membentuk representasi baru berdasarkan bobot perhatian tersebut

Contoh Kalimat:

- “Bank di tepi sungai itu indah.”

Kata “Bank” dapat berarti:

- Lembaga keuangan
- Tepi sungai

Dengan self-attention, model melihat kata “sungai” sehingga memahami makna yang benar.

1.3.4 4. Representasi Q,K,V (Query, Key, Value)

Setiap token dalam kalimat diubah menjadi tiga representasi:

- 1) Query (Q) → Apa yang saya cari?
- 2) Key (K) → Apa yang saya tawarkan?
- 3) Value (V) → Informasi yang saya bawa

Hasil dari perhitungan QKV adalah representasi baru setiap kata yang mempertimbangkan seluruh konteks kalimat.

1.3.5 5. Multi-Head Attention

Alih-alih menghitung satu attention saja, Transformer menggunakan beberapa attention secara paralel.

Mengapa?

Karena setiap “head” dapat menangkap jenis hubungan berbeda:

- 1) Hubungan subjek–predikat
- 2) Hubungan kata sifat–kata benda
- 3) Hubungan temporal
- 4) Hubungan sebab-akibat

Multi-head attention memungkinkan model memahami berbagai pola linguistik secara simultan.

6. Positional Encoding Karena Transformer tidak membaca secara berurutan seperti RNN, ia memerlukan informasi posisi kata.

Solusinya adalah Positional Encoding, yaitu menambahkan informasi posisi ke embedding token.

Tanpa positional encoding:

- “Saya makan nasi”
- “Nasi makan saya”

Akan terlihat identik bagi model.

Positional encoding memastikan urutan tetap bermakna.

1.3.6 7. Struktur Arsitektur Transformer

Transformer terdiri dari dua bagian utama:

1) Encoder

- Digunakan untuk memahami input
- Terdiri dari: 1) Multi-Head Self Attention, 2) Feed Forward Network, 3) Residual connection, 4) Layer normalization

2) Decoder

- Digunakan untuk menghasilkan output
- Digunakan dalam tugas seperti machine translation

Model seperti: - BERT → hanya menggunakan Encoder - GPT → hanya menggunakan Decoder

1.3.7 8. Ringkasan

Transformer menggantikan mekanisme memori berantai (RNN) dengan mekanisme perhatian global (self-attention). Dengan pendekatan ini, setiap kata dapat secara langsung berinteraksi dengan semua kata lain dalam kalimat, menghasilkan pemahaman konteks yang lebih kuat dan fleksibel.

Inilah yang membuat Transformer menjadi arsitektur revolusioner dalam NLP modern.

1.4 Praktikum

1.4.1 Praktikum 1: Konsep Self-Attention

Self-Attention memungkinkan model untuk melihat hubungan antar kata dalam sebuah kalimat dengan cara:

- 1) Mengubah kata menjadi vektor (Embedding)
- 2) Menghitung Query (Q), Key (K), dan Value (V)
- 3) Menghitung Attention Scores menggunakan dot product antara Q dan K
- 4) Menerapkan Softmax untuk normalisasi
- 5) Menghitung Self-Attention Output dengan mengalikan bobot hasil Softmax dengan Value (V)

Berikut adalah contoh sederhananya. Asumsikan kita memiliki 3 kata: “Aku”, “suka”, “apel”
Setiap kata diubah menjadi vektor numerik untuk simulasi embedding.

```
[37]: import numpy as np
import pandas as pd

#1 Representasi kata sebagai vektor sederhana (embedding)
words = ["Aku", "suka", "apel"]
X = np.array([
    [1, 0, 1], # "Aku"
```

```

    [0, 1, 1], # "suka"
    [1, 1, 0]  # "apel"
])

```

Langkah selanjutnya adalah membuat matriks bobot Q,K,V

- Matriks Query (Q) menentukan bagaimana setiap kata mencari informasi dari kata lain.
- Matriks Key (K) menentukan bagaimana kata-kata menyediakan informasi bagi yang mencarinya.
- Matriks Value (V) menentukan representasi informasi dari kata itu sendiri.

[30]: *#2 Matriks bobot untuk Query (Q), Key (K), dan Value (V)*

```

W_Q = np.array([[1, 0, 1],
                 [0, 1, 1],
                 [1, 1, 0]])

W_K = np.array([[0, 1, 1],
                 [1, 1, 0],
                 [1, 0, 1]])

W_V = np.array([[1, 1, 0],
                 [1, 0, 1],
                 [0, 1, 1]])

```

Berikutnya adalah Menggunakan dot product antara embedding dan matriks bobot.

[31]: *#3 Menghitung Query (Q), Key (K), dan Value (V)*

```

Q = np.dot(X, W_Q)
K = np.dot(X, W_K)
V = np.dot(X, W_V)

```

Attention Scores dihitung dengan dot product antara Q dan K transpose (K^T). Ini menghasilkan bobot kasar tentang hubungan antar kata.

[32]: *#4 Menghitung Attention Scores ($Q * K^T$)*

```

attention_scores = np.dot(Q, K.T)

```

Selanjutnya menerapkan Softmax. Softmax memastikan bobot antar kata tetap dalam skala probabilitas (0-1) dan jumlahnya 1 dalam satu baris.

[33]: *#5 Menerapkan Softmax pada Attention Scores*

```

def softmax(x):
    exp_x = np.exp(x - np.max(x)) # Stabilitas numerik
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

attention_weights = softmax(attention_scores)

```

Kemudian, perhitungan Attention Weights. Attention Weights dikalikan dengan Value (V) untuk mendapatkan representasi akhir dari setiap kata setelah diperhitungkan dengan konteksnya.

```
[36]: # 6) Menghitung Self-Attention Output (AttentionWeights * V)
attention_output = np.dot(attention_weights, V)
```

Langkah selanjutnya adalah menampilkan hasil dalam bentuk matrix seperti berikut:

```
[38]: # 7) Menampilkan Hasil dalam bentuk DataFrame
df_attention_scores = pd.DataFrame(attention_scores, columns=words, index=words)
df_attention_weights = pd.DataFrame(attention_weights, columns=words,
    ↪ index=words)
df_attention_output = pd.DataFrame(attention_output, columns=["Dim 1", "Dim 2",
    ↪ "Dim 3"], index=words)

print("\n Q (Query):")
print(pd.DataFrame(Q, columns=["Dim 1", "Dim 2", "Dim 3"], index=words))

print("\n K (Key):")
print(pd.DataFrame(K, columns=["Dim 1", "Dim 2", "Dim 3"], index=words))

print("\n V (Value):")
print(pd.DataFrame(V, columns=["Dim 1", "Dim 2", "Dim 3"], index=words))

print("\n Attention Scores (Q * K^T):")
print(df_attention_scores)

print("\n Self-Attention Weights (Softmax dari Attention Scores):")
print(df_attention_weights)

print("\n Self-Attention Output (AttentionWeights * V):")
print(df_attention_output)
```

```
Q (Query):
      Dim 1  Dim 2  Dim 3
Aku        2      1      1
suka       1      2      1
apel       1      1      2
```

```
K (Key):
      Dim 1  Dim 2  Dim 3
Aku        1      1      2
suka       2      1      1
apel       1      2      1
```

```
V (Value):
      Dim 1  Dim 2  Dim 3
Aku        1      2      1
suka       1      1      2
apel       2      1      1
```

Attention Scores ($Q * K^T$):

	Aku	suka	apel
Aku	5	6	5
suka	5	5	6
apel	6	5	5

Self-Attention Weights (Softmax dari Attention Scores):

	Aku	suka	apel
Aku	0.211942	0.576117	0.211942
suka	0.211942	0.211942	0.576117
apel	0.576117	0.211942	0.211942

Self-Attention Output ($\text{AttentionWeights} * V$):

	Dim 1	Dim 2	Dim 3
Aku	1.211942	1.211942	1.576117
suka	1.576117	1.211942	1.211942
apel	1.211942	1.576117	1.211942

Berikut adalah penjelasannya:

- 1) Self-Attention Weights menunjukkan bobot perhatian yang diberikan setiap kata terhadap kata lainnya.
- 2) Self-Attention Output menunjukkan hasil akhir setelah attention dihitung dengan Value (V).

Interpretasi:

- Kata “Aku” lebih memperhatikan kata “suka” (0.5761) dibandingkan dirinya sendiri (0.2119) atau “apel” (0.2119).
- Kata “suka” membagi perhatian secara relatif seimbang, tetapi lebih fokus ke “apel” (0.5761).
- Kata “apel” lebih memperhatikan “Aku” (0.5761), bukan “suka”.

Interpretasi Self-Attention Output:

- Kata “Aku” memiliki nilai terbesar pada Dim 3 (1.5761). Ini menunjukkan bahwa representasi “Aku” setelah Self-Attention banyak dipengaruhi oleh informasi yang ada di Dim 3. Nilai ini bisa datang dari pengaruh kata “suka” dan “apel” yang memberikan bobot besar pada “Aku”.
- Kata “Suka” memiliki nilai terbesar pada Dim 1 (1.5761). Artinya “suka” setelah Self-Attention lebih dipengaruhi oleh informasi dalam Dim 1. Nilai ini bisa datang dari hubungan dengan “apel” yang memberinya bobot perhatian besar.
- Kata “Apel” memiliki nilai terbesar pada Dim 2 (1.5761). Artinya “apel” setelah Self-Attention lebih dipengaruhi oleh informasi dalam Dim 2. Nilai ini bisa datang dari hubungan dengan “Aku” atau “suka”.

Jika suatu kata memiliki nilai terbesar dalam suatu dimensi, artinya dimensi tersebut lebih banyak mengandung informasi dari kata tersebut. Nilai terbesar dalam suatu dimensi mencerminkan pengaruh terbesar dari kata tertentu dalam hasil Self-Attention.

1.4.2 Praktikum 2: Menggunakan Pre-Trained BERT

Pada praktikum pertama ini, mahasiswa akan memahami bagaimana BERT menghasilkan representasi numerik (embedding) dari sebuah kalimat.

Berbeda dengan Word2Vec yang menghasilkan embedding tetap (fixed embedding), BERT menghasilkan contextual embedding, artinya vektor kata akan berubah tergantung konteks kalimat.

Fokus praktikum ini:

- 1) Memahami proses tokenisasi BERT
- 2) Mengamati struktur input [CLS] dan [SEP]
- 3) Mengambil embedding representasi kalimat dari token [CLS]

1. Install Library

```
[3]: !pip install transformers
      !pip install torch
```

```
Requirement already satisfied: transformers in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (5.2.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (1.4.1)
Requirement already satisfied: numpy>=1.17 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (2025.9.1)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (0.22.2)
Requirement already satisfied: typer-slim in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (0.21.1)
Requirement already satisfied: safetensors>=0.4.3 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
transformers) (4.67.1)
Requirement already satisfied: filelock in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (3.24.3)
```


Requirement already satisfied: fsspec>=2023.5.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (2026.2.0)

Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (1.3.1)

Requirement already satisfied: httpx<1,>=0.23.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (0.28.1)

Requirement already satisfied: shellingham in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (1.5.4)

Requirement already satisfied: typing-extensions>=4.1.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
huggingface-hub<2.0,>=1.3.0->transformers) (4.15.0)

Requirement already satisfied: colorama in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
tqdm>=4.27->transformers) (0.4.6)

Requirement already satisfied: click>=8.0.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
typer-slim->transformers) (8.2.1)

Requirement already satisfied: anyio in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (4.9.0)

Requirement already satisfied: certifi in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (2025.4.26)

Requirement already satisfied: httpcore==1.* in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)

Requirement already satisfied: idna in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (3.10)

Requirement already satisfied: h11>=0.16 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers)
(0.16.0)

Requirement already satisfied: sniffio>=1.1 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
anyio->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (1.3.1)

[notice] A new release of pip is available: 24.0 -> 26.0.1

[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: torch in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages
(2.10.0)

Requirement already satisfied: filelock in

```

c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (3.24.3)
Requirement already satisfied: typing-extensions>=4.10.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (3.6.1)
Requirement already satisfied: jinja2 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
torch) (2026.2.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
c:\users\rani\appdata\local\programs\python\python311\lib\site-packages (from
jinja2->torch) (3.0.2)

```

[notice] A new release of pip is available: 24.0 -> 26.0.1

[notice] To update, run: python.exe -m pip install --upgrade pip

Blok kode ini digunakan untuk menginstal library utama yang diperlukan. transformers menyediakan model dan tokenizer BERT yang sudah pre-trained, sedangkan torch berfungsi sebagai mesin komputasi tensor dan neural network yang menjalankan model BERT. Tanpa dua library ini, model tidak dapat dijalankan.

2. Import and Load Model

```

[1]: from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
model.eval()

```

```

c:\Users\rani\AppData\Local\Programs\Python\Python311\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Loading weights: 100%|          | 199/199 [00:01<00:00, 150.67it/s,
Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased

```

Key	Status		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[1]: BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (intermediate): BertIntermediate(
      (dense): Linear(in_features=768, out_features=3072, bias=True)
      (intermediate_act_fn): GELUActivation()
    )
    (output): BertOutput(
      (dense): Linear(in_features=3072, out_features=768, bias=True)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
)
```

```

    )
    )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
)

```

Pada tahap ini kita memuat tokenizer dan model BERT yang sudah dilatih sebelumnya. Tokenizer bertugas mengubah teks menjadi format numerik yang dapat dipahami model, sedangkan Bert-Model menghasilkan representasi embedding. Perintah `model.eval()` mengaktifkan mode evaluasi agar model tidak menggunakan mekanisme seperti dropout, sehingga hasil embedding menjadi stabil.

3. Tokenisasi Input

```

[2]: text = "I love studying Natural Language Processing"
     inputs = tokenizer(text, return_tensors='pt')
     print(inputs)

```

```

{'input_ids': tensor([[ 101, 1045, 2293, 5702, 3019, 2653, 6364,  102]]),
 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask':
 tensor([[1, 1, 1, 1, 1, 1, 1, 1]])}

```

Kode di atas mengubah kalimat menjadi input numerik berupa `input_ids` dan `attention_mask`. Tokenizer secara otomatis menambahkan token khusus seperti [CLS] dan [SEP]. Parameter `return_tensors='pt'` memastikan output berbentuk tensor PyTorch sehingga bisa langsung diproses oleh model.

4. Forward Pass Model

```

[3]: with torch.no_grad():
     outputs = model(**inputs)

     last_hidden_state = outputs.last_hidden_state
     print(last_hidden_state.shape)

```

```

torch.Size([1, 8, 768])

```

Blok di atas menjalankan kalimat melalui model BERT untuk menghasilkan embedding. Penggunaan `torch.no_grad()` bertujuan menonaktifkan perhitungan gradien karena kita hanya melakukan inference, bukan training. Output utama berupa `last_hidden_state`, yaitu representasi embedding setiap token dalam kalimat dengan dimensi 768 untuk BERT-base.

5. Mengambil Embedding Kalimat ([CLS])

```

[4]: cls_embedding = last_hidden_state[:, 0, :]
     print(cls_embedding.shape)

```

```

torch.Size([1, 768])

```

Kode di atas mengambil vektor pada posisi pertama, yaitu token [CLS], yang secara umum digunakan sebagai representasi keseluruhan kalimat. Vektor inilah yang biasanya dipakai untuk tugas klasifikasi teks atau analisis semantik tingkat kalimat.

1.4.3 Praktikum 3: Contextual Embedding

1. Definisi Kalimat

```
[5]: kalimat1 = "He went to the bank to deposit money"
    kalimat2 = "He sat on the river bank"
```

Dua kalimat ini digunakan untuk menunjukkan bahwa kata “bank” memiliki makna berbeda tergantung konteks. Tujuan eksperimen ini adalah membuktikan bahwa BERT menghasilkan embedding berbeda untuk kata yang sama jika konteksnya berubah.

2. Fungsi Ekstraksi Embedding

```
[10]: def get_word_embedding(sentence, target_word):
        inputs = tokenizer(sentence, return_tensors='pt')
        with torch.no_grad():
            outputs = model(**inputs)
        tokens = tokenizer.tokenize(sentence)
        index = tokens.index(target_word)
        return outputs.last_hidden_state[0, index+1, :]

emb1 = get_word_embedding(kalimat1, "bank")
emb2 = get_word_embedding(kalimat2, "bank")
```

Fungsi di atas digunakan untuk mengambil embedding sebuah kata tertentu dalam kalimat. Kalimat diproses melalui tokenizer dan model, lalu diambil vektor token yang sesuai. Penambahan +1 dilakukan karena posisi awal sudah ditempati token [CLS].

3. Hitung Similarity

```
[11]: from sklearn.metrics.pairwise import cosine_similarity
    from sklearn.metrics.pairwise import cosine_similarity
    cosine_similarity([emb1], [emb2])
```

```
[11]: array([[0.32483876]], dtype=float32)
```

Cosine similarity digunakan untuk mengukur tingkat kemiripan antara dua vektor embedding. Jika nilainya rendah, berarti BERT membedakan makna kata berdasarkan konteks. Inilah bukti bahwa BERT menghasilkan contextual embedding.

1.4.4 Praktikum 4: Masked Language Modeling

1. Load Model MLM

```
[13]: from transformers import BertForMaskedLM
    model_mlm = BertForMaskedLM.from_pretrained('bert-base-uncased')
    model_mlm.eval()
```

```
Loading weights: 100%|          | 202/202 [00:00<00:00, 972.72it/s,
Materializing param=cls.predictions.transform.dense.weight]
```

BertForMaskedLM LOAD REPORT from: bert-base-uncased

Key	Status		
bert.pooler.dense.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
bert.pooler.dense.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
[13]: BertForMaskedLM(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
  )
)
```

```

    )
    )
)
(cls): BertOnlyMLMHead(
  (predictions): BertLMPredictionHead(
    (transform): BertPredictionHeadTransform(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (transform_act_fn): GELUActivation()
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    )
    (decoder): Linear(in_features=768, out_features=30522, bias=True)
  )
)
)
)

```

Model ini merupakan versi BERT yang dilengkapi kepala prediksi untuk tugas Masked Language Modeling. Model ini digunakan untuk menebak kata yang disembunyikan dalam sebuah kalimat.

2. Input dengan Mask

```
[14]: text = "Today I will study [MASK] processing"
      inputs = tokenizer(text, return_tensors='pt')
```

Kalimat di atas mengandung token [MASK], yaitu kata yang akan ditebak oleh model berdasarkan konteks kalimat secara dua arah.

3. Prediksi

```
[15]: with torch.no_grad():
      outputs = model_mlm(**inputs)

      logits = outputs.logits
```

Model memproses input dan menghasilkan logits, yaitu skor probabilitas untuk setiap kata dalam vocabulary. Nilai tertinggi pada posisi [MASK] akan menjadi prediksi kata yang dianggap paling sesuai.

1.4.5 Praktikum 5: FineTuning untuk Klasifikasi

Pada praktikum ini mahasiswa akan mempraktikkan konsep penting: pre-trained model tidak langsung “paham” tugas spesifik kita. BERT sudah dilatih pada tugas umum bahasa (MLM/NSP), sehingga agar mampu melakukan klasifikasi sentimen, kita perlu melakukan fine-tuning. Fine-tuning berarti melatih ulang model dengan dataset berlabel, biasanya dengan learning rate kecil, sehingga representasi BERT menyesuaikan pola dari tugas klasifikasi yang kita inginkan.

1. Import Library

```
[20]: import torch
      from transformers import BertTokenizer, BertForSequenceClassification
      from torch.optim import AdamW
      from sklearn.metrics import accuracy_score, classification_report
```

Kita memanggil PyTorch untuk tensor dan training, HuggingFace untuk tokenizer dan model klasifikasi, optimizer AdamW untuk fine-tuning, serta metrik evaluasi untuk melihat performa model.

2. Membuat Dataset Contoh

```
[21]: texts = [
    "I love this product",
    "This is terrible",
    "Very good service",
    "I hate this",
    "Amazing experience",
    "Worst service ever"
]

# 1 = positif, 0 = negatif
labels = torch.tensor([1, 0, 1, 0, 1, 0])
```

Dataset di atas adalah sample dataset kecil untuk fokus pada alur fine-tuning. Label dibuat sebagai tensor karena model klasifikasi BERT membutuhkan label dalam format tensor PyTorch.

3. Load Tokenizer

```
[23]: tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model_cls = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
Loading weights: 100%|          | 199/199 [00:00<00:00, 837.53it/s,
Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                     | Status      |
-----+-----+
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.seq_relationship.bias                 | UNEXPECTED |
cls.predictions.transform.dense.weight    | UNEXPECTED |
cls.predictions.transform.dense.bias      | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.bias                      | UNEXPECTED |
classifier.weight                         | MISSING    |
classifier.bias                           | MISSING    |
```

Notes:

- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING : those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Tokenizer mengubah teks menjadi token/ID sesuai vocabulary BERT. Model hanya bisa bekerja pada input numerik, bukan string mentah. Kita memuat BERT yang sudah ditambah “kepala

klasifikasi” (classification head). `num_labels=2` berarti output model memiliki dua kelas (positif/negatif).

4. Tokenisasi Batch (Padding & Truncation)

```
[24]: inputs = tokenizer(  
    texts,  
    padding=True,  
    truncation=True,  
    return_tensors="pt"  
)
```

Tokenisasi batch membuat semua teks punya panjang input yang seragam (padding) dan memotong kalimat terlalu panjang (truncation). Output berupa `input_ids` dan `attention_mask` dalam bentuk tensor.

5. Buat Optimizer AdamW

```
[25]: optimizer = AdamW(model_cls.parameters(), lr=2e-5)
```

Fine-tuning umumnya memakai learning rate kecil (misalnya $2e-5$) karena model sudah “pintar” secara umum. Learning rate besar bisa merusak pengetahuan pre-trained.

6. Finetuning

1) Training

```
[26]: model_cls.train()  
  
epochs = 3  
for epoch in range(epochs):  
    optimizer.zero_grad()  
  
    outputs = model_cls(**inputs, labels=labels)  
    loss = outputs.loss  
  
    loss.backward()  
    optimizer.step()  
  
    print(f"Epoch {epoch+1}/{epochs} | Loss: {loss.item():.4f}")
```

Epoch 1/3 | Loss: 0.6984

Epoch 2/3 | Loss: 0.6622

Epoch 3/3 | Loss: 0.6127

Di atas adalah kode inti fine-tuning. Model menghitung loss dari prediksi vs label, lalu `backward()` menghitung gradien, dan `optimizer.step()` memperbarui parameter. `zero_grad()` wajib untuk mencegah gradien menumpuk dari iterasi sebelumnya.

2) Evaluasi dan Prediksi Kelas

```
[27]: model_cls.eval()

with torch.no_grad():
    outputs = model_cls(**inputs)
    logits = outputs.logits
    preds = torch.argmax(logits, dim=1)
```

Setelah training, kita masuk mode evaluasi agar dropout dimatikan. `no_grad()` membuat proses lebih ringan. Prediksi kelas diambil dari `argmax` logit.

3) Hitung akurasi dan laporan klasifikasi

```
[28]: y_true = labels.detach().cpu().numpy()
y_pred = preds.detach().cpu().numpy()

print("Accuracy:", accuracy_score(y_true, y_pred))
print(classification_report(y_true, y_pred))
```

Accuracy: 0.8333333333333334

	precision	recall	f1-score	support
0	0.75	1.00	0.86	3
1	1.00	0.67	0.80	3
accuracy			0.83	6
macro avg	0.88	0.83	0.83	6
weighted avg	0.88	0.83	0.83	6

Kita pindahkan hasil ke CPU untuk dihitung metriknya menggunakan `sklearn`. `classification_report` memberi ringkasan precision, recall, dan f1-score per kelas.